Day 140: Reverse Diffusion From Scratch: a Real Denoising Step

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 1 | Complete study guide

Watch: Diffusion Models: DDPM | Generative AI Animated (Deepia) | Diffusion Models | Paper Explanation | Math Explained (Outlier)

Script: `day140_reverse_diffusion.py` · every number in this guide is copied by program from `day140_run_output.txt`, the real output of that script.

Day 139 destroyed data with 1,000 small steps of noise. Today runs that process backwards. On Day 135's ring of eight Gaussians the best possible denoiser can be written down exactly, so two questions are kept apart: is the reverse process itself right, and how good is a learned denoiser? With the exact denoiser, 1,000 steps back from pure noise recovered all eight modes (good share 0.998, 0.995, 0.997) but slightly too tight (spread 0.0455 against the real ring's 0.0509). A small network trained for 30,000 steps came within 0.0129 of the lowest possible error and also recovered eight modes, slightly too wide (spread 0.0607). Without fresh noise at each step, the learned denoiser's samples fell into three of the eight modes (counts 1059 366 0 0 0 0 0 571).

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

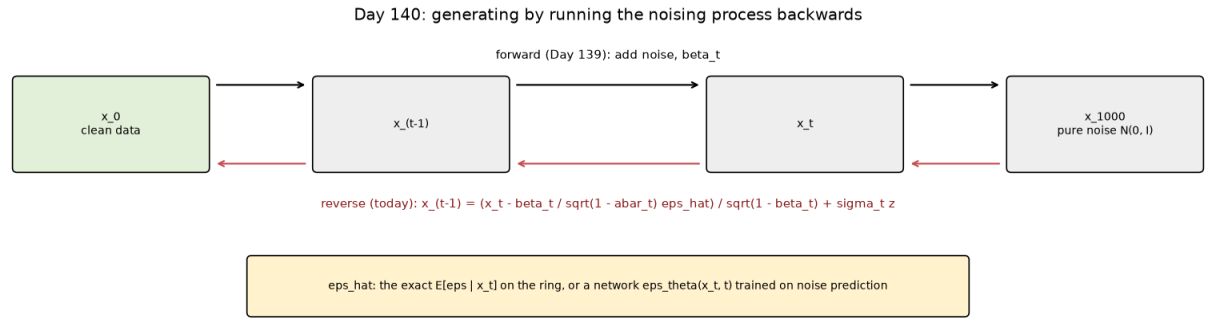
A **diffusion model** generates data by undoing noise. The **forward process** (Day 139) adds a little Gaussian noise at each of T = 1,000 steps until nothing of the data is left: x_{1000} is pure N(0, I) noise. The **reverse process** starts from fresh noise and takes 1,000 small **denoising steps**, each from x_t to a slightly cleaner x_{t-1}.

Every step needs one thing: a guess of the noise ε hidden inside x_t (equivalently, a guess of the clean x_0). That guess is the **denoiser**. In a real model it is a neural network $\varepsilon_\theta(x_t, t)$; on the ring it can also be computed exactly, which gives the network a target and a floor.

REAL-WORLD SCENARIO

Restoring a photograph that has faded through a thousand small exposures to sunlight. No restorer can undo all the fading in one stroke, because a completely faded print could have been any photograph. But each single exposure changed it only a little, so it can be undone a little at a time, with the restorer's knowledge of what photographs usually look like guiding every step.

SYSTEM DIAGRAM



Forward: Day 139's noising. Reverse: today's denoising step, using a guess of the noise.

Today's part for the week's build

`day140_exercise.py` is the reverse process (the posterior, the exact denoiser on the ring, the sampling loop, the time embedding and the noise-predicting network), checked by hand: the posterior at both ends of the schedule, one exact step from t = 20, the whole reverse process with the exact denoiser, and a network trained for only 3,000 steps, which recovers eight modes but far too spread out.

ACTUAL OUTPUT

```
posterior at t = 1: x_0 weight 1.0000, x_t weight 0.0000, variance 0.000000 (step 1 starts from
    x_0 itself)
posterior at t = 1,000: x_0 weight 0.000128, x_t weight 0.9899, variance 0.020000
one exact step from t = 20: distance to the true x_0 0.0512; modes 8
reverse process, exact denoiser, noise beta_t: 8 modes, good share 0.989, spread 0.0497 (real
    0.05)
after 3,000 training steps, the learned denoiser: 8 modes, good share 0.636, spread 0.1490
```

Data: Day 135's ring, eight Gaussians of standard deviation 0.05 at radius 2. The noise schedule (linear β from 0.0001 to 0.02 over 1,000 steps), q_sample, q_step, the ring and Day 135's mode count are Day 139's parts; the network, its training pass and Adam are Day 126's. All are imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: one reverse step, given the clean data

DEFINITION — start here, no prior knowledge assumed

If the clean x_0 is known, the step back from x_t to x_{t-1} has an exact answer, the **posterior** $q(x_{t-1} \mid x_t, x_0)$: a Gaussian whose mean mixes a little of x_0 with most of x_t, and whose variance V_t is a little below β_t. Every diffusion sampler is built from this formula, so it is checked first: a million forward runs, and a straight-line fit of x_{t-1} on x_t.

REAL-WORLD SCENARIO

Knowing where a walker started and where they are now: their position one step ago is fairly predictable, mostly near where they are now, pulled slightly back towards the start.

MATHEMATICS

$$q(x_{t-1} \mid x_t, x_0) = N(c_0\, x_0 + c_t\, x_t,\ V_t), \quad A_t = \prod_{s \le t}(1-\beta_s) \ \text{(ABAR in the code)}$$

$$c_0 = \sqrt{A_{t-1}}\,\beta_t / (1-A_t), \quad c_t = \sqrt{(1-\beta_t)}\,(1-A_{t-1}) / (1-A_t), \quad V_t = (1-A_{t-1}) / (1-A_t) \cdot \beta_t$$

V_t is called beta-tilde in the code and the output; ε' and x'_0 below mean guesses of ε and x_0

IMPLEMENTATION

```
def posterior(t):
    b, ab, ab1 = BETAS[t - 1], ABAR[t], ABAR[t - 1]
    c0 = np.sqrt(ab1) * b / (1 - ab)
    ct = np.sqrt(1 - b) * (1 - ab1) / (1 - ab)
    return c0, ct, (1 - ab1) / (1 - ab) * b
```

ACTUAL OUTPUT

```
x_0 = 1.0; a million runs of the forward process to step t - 1, then one more step; x_(t-1)
    regressed on x_t
     t    c0 x_0  simulated        ct  simulated  beta-tilde  simulated
     2   0.54529    0.54476   0.45471    0.45525    0.000055   0.000054
    50   0.03660    0.03634   0.96339    0.96362    0.001037   0.001035
   300   0.00634    0.00619   0.99298    0.99305    0.006032   0.006029
   900   0.00030    0.00033   0.99095    0.99095    0.018008   0.017987
given x_0, x_(t-1) and x_t are jointly Gaussian, so a straight-line fit recovers the posterior
    exactly (up to sampling error)
Actual output from this lesson's run.
```

What it means

Formula and simulation agree at every t. At t = 50 the weight on x_0 is 0.03660 (simulated 0.03634), the weight on x_t 0.96339 (0.96362), and the variance left 0.001037 (0.001035). Early steps lean on x_0 (t = 2: 0.54529); late steps keep almost all of x_t. The catch: x_0 is exactly what a generator does not know. Every reverse step therefore has to replace x_0 with a guess.

WHY CHOOSE THIS?

A closed form: one formula serves every step and every sampler.

WHY NOT THIS? (TRADEOFFS)

It needs x_0; the whole problem of generation is moved into guessing it well.

2. One denoising step: the best possible guess of x_0

DEFINITION — start here, no prior knowledge assumed

The best guess of x_0 from x_t, in squared error, is the average over every clean point that could have produced it: $E[x_0 \mid x_t]$. On the ring this is exact: Bayes' rule says how likely each of the eight modes is, and within a mode the guess is a Gaussian conditional mean. 2,000 ring points are noised to step t and then guessed back in one go.

REAL-WORLD SCENARIO

Guessing a blurred house number. Slightly blurred, you read it exactly. Heavily blurred, the best you can do is 'probably a 3 or an 8', and if forced to name one number you would pick something between them that is neither.

MATHEMATICS

given mode j: $x_t \sim N(\sqrt{A_t}\, C_j,\ v\, I)$, $\quad v = A_t\, \sigma^2 + 1 - A_t$, $\quad \sigma = 0.05$

$p(j \mid x_t) \propto exp(-|x_t - \sqrt{A_t}\, C_j|^2 / 2v)$, $\quad E[x_0 \mid x_t] = \Sigma_j\, p(j \mid x_t)\, [C_j + (\sqrt{A_t}\, \sigma^2 / v)(x_t - \sqrt{A_t}\, C_j)]$

the noise it implies: $\varepsilon' = (x_t - \sqrt{A_t}\, x'_0) / \sqrt{(1 - A_t)}$

IMPLEMENTATION

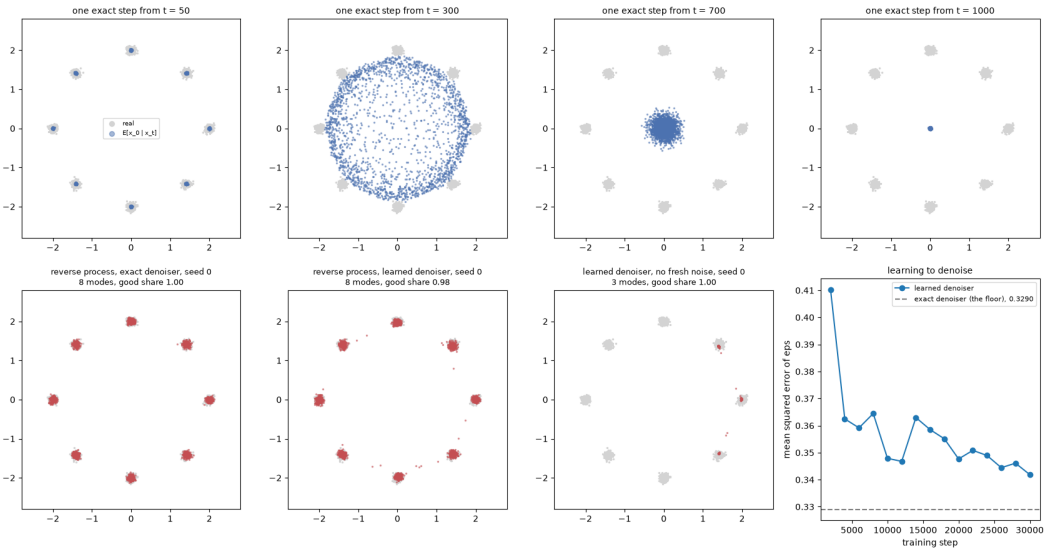
```
def exact_x0(xt, t, radius=2.0, sd=0.05, k=8):
    C = np.c_[np.cos(2 * np.pi * np.arange(k) / k), np.sin(2 * np.pi * np.arange(k) / k)] * radius
    a, v = np.sqrt(ABAR[t]), ABAR[t] * sd ** 2 + 1 - ABAR[t]
    L = -((xt[:, None, :] - a * C[None]) ** 2).sum(-1) / (2 * v)
    w = np.exp(L - L.max(1, keepdims=True)); w /= w.sum(1, keepdims=True)
    m = C[None] + (a * sd ** 2 / v) * (xt[:, None, :] - a * C[None])
    return (w[..., None] * m).sum(1)
```

ACTUAL OUTPUT

```
2,000 ring points noised to step t, then the exact guess of x_0 from x_t alone; Day 135's
    mode_stats on the guesses
     t  noise sd  modes  good share  guess radius  |guess - x_0|  right mode
     1    0.0100      8       0.992        1.9990         0.0123       1.000
    10    0.0435      8       1.000        1.9995         0.0408       1.000
    50    0.1702      8       1.000        1.9996         0.0602       1.000
   100    0.3209      8       0.916        1.9883         0.1100       0.978
   200    0.5839      8       0.203        1.8526         0.5673       0.722
   300    0.7769      0       0.003        1.5366         1.0436       0.475
   500    0.9599      0       0.000        0.7022         1.7823       0.231
   700    0.9965      0       0.000        0.2101         1.9771       0.148
  1000    1.0000      0       0.000        0.0160         1.9993       0.123
Actual output from this lesson's run.
```

What it means

Up to t = 50 one step is enough: the guesses sit on the eight modes, 0.0602 from the true x_0 on average. By t = 200 only 0.203 of the guesses land on a mode and the guesses shrink inwards (average radius 1.8526): when two modes are both plausible, the average falls between them. At t = 1,000 every guess is the centre of the ring (radius 0.0160) and the right mode is named 0.123 of the time, chance for eight modes. This is why a diffusion model takes many small steps: a single jump from pure noise can only produce the average of all data, a blur. Each small step needs only a local correction, and the next step's guess is made from a less noisy x_t.



Top: one exact denoising step from t = 50, 300, 700 and 1,000 (blue) against the real ring (grey). Bottom: 1,000 steps with the exact denoiser; 1,000 steps with the learned one; the learned one with no fresh noise; the learned denoiser's error during training.

WHY CHOOSE THIS?

The posterior mean is the best guess in squared error, the quantity a network is trained to approximate.

WHY NOT THIS? (TRADEOFFS)

It is an average, not a sample: used alone from high noise it returns a blur.

3. The whole reverse process with the exact denoiser

DEFINITION — start here, no prior knowledge assumed

Start from 2,000 points of pure noise and apply Ho et al.'s (2020) reverse step 1,000 times, each time using the exact guess of the noise and adding fresh Gaussian noise of variance V_t. If the reverse process is right, the eight modes come back in equal shares with the ring's own spread, 0.05.

REAL-WORLD SCENARIO

A road map drawn perfectly: if the traveller still ends up in the wrong town, the directions (the process) are wrong, not the map.

MATHEMATICS

$$x_{t-1} = (x_t - \beta_t / \sqrt{(1 - A_t)} \cdot \varepsilon'(x_t, t)) / \sqrt{(1 - \beta_t)} + \sigma_t z, \quad z \sim N(0, I), \quad \text{no } z \text{ at } t = 1$$

the mean term is the posterior mean of section 1 with x_0 replaced by its guess

IMPLEMENTATION

```
def reverse_sample(eps_fn, n, rng, var="posterior", d=2):
    x = rng.standard_normal((n, d))
    for t in range(len(BETAS), 0, -1):
        b = BETAS[t - 1]; e = eps_fn(x, t)
        x = (x - b / np.sqrt(1 - ABAR[t]) * e) / np.sqrt(1 - b)
        if t > 1:
            s2 = posterior(t)[2] if var == "posterior" else (b if var == "beta" else 0.0)
            x = x + np.sqrt(s2) * rng.standard_normal(x.shape)
    return x
```

ACTUAL OUTPUT

```
real ring, 2,000 points: 8 modes, good share 0.982, spread around its centre 0.0509, counts 237 268 229 234
    233 244 270 249
2,000 samples, 1,000 steps from N(0, I), the exact eps at every step, fresh noise of variance beta-tilde_t:
   seed 0: 8 modes, good share 0.998, spread 0.0455, counts 232 239 266 244 250 261 263 240 (0.8 s)
   seed 1: 8 modes, good share 0.995, spread 0.0457, counts 247 231 252 252 236 268 237 266 (0.8 s)
   seed 2: 8 modes, good share 0.997, spread 0.0460, counts 241 219 226 285 240 253 256 273 (1.0 s)
Actual output from this lesson's run.
```

What it means

All eight modes come back in all three seeds, in roughly equal shares (the smallest count 219, the largest 285, against 250 each if exactly even). The samples are a little too tight: spread 0.0455 to 0.0460 against the real ring's 0.0509. A likely reason: the true step back from x_t is a mixture over every x_0 that could have produced it, so its variance is V_t plus c_0^2 times the uncertainty left in x_0; adding only V_t leaves that second part out. Section 5 tests the other textbook choice, β_t.

WHY CHOOSE THIS?

With an exact denoiser the reverse process itself can be judged with nothing learned in the way.

WHY NOT THIS? (TRADEOFFS)

Exact denoisers exist only for toy data like this ring; for images the network is the only option.

4. A learned denoiser, against the lowest possible error

DEFINITION — start here, no prior knowledge assumed

A network $\varepsilon_\theta(x_t, t)$ is trained the way real diffusion models are: take a clean point, pick a random step t and random noise ε, form x_t with Day 139's shortcut, and penalise the squared error between ε and the network's guess. The step number enters as 32 sines and cosines. Its error is measured on 20,000 fixed (x_0, t, ε) triples, against the exact denoiser's error on the same triples: the floor no network can go below.

REAL-WORLD SCENARIO

A trainee radiologist's error rate measured against the best possible rate: some scans are ambiguous to anyone, so the target is the expert's error, not zero.

MATHEMATICS

$$loss = mean\ |\varepsilon - \varepsilon_\theta(\sqrt{A_t}\, x_0 + \sqrt{(1 - A_t)}\, \varepsilon,\ t)|^2, \quad t\ uniform\ on\ 1–1{,}000$$

$$time\ embedding: [sin(t\, f_i), cos(t\, f_i)],\ f_i = 10000^{-i/16},\ i = 0..15$$

SPECIFICATION

Network	[x_t (2), time embedding (32)] → 256 → 256 → 2, ReLU; Day 126's init_net, forward, backward, Adam (lr 0.001)
Training	30,000 steps, batches of 512 fresh ring points
Evaluated	every 2,000 steps on 20,000 fixed triples, t in 9 levels

IMPLEMENTATION

```
def train_denoiser(steps, seed=0, batch=512, lr=1e-3, every=2000, Xev=None):
    net = d126.init_net([34, 256, 256, 2], seed); opt = d126.Adam(net, lr); acts = ["relu",
        "relu", "linear"]
    rng = np.random.RandomState(seed); log = []
    for s in range(1, steps + 1):
        x0 = d139.ring(batch, rng).astype(np.float64); t = rng.randint(1, len(BETAS) + 1, batch)
        e = rng.standard_normal(x0.shape); xt = np.sqrt(ABAR[t])[:, None] * x0 + np.sqrt(1 -
            ABAR[t])[:, None] * e
        A = d126.forward(net, np.concatenate([xt, time_embed(t)], 1).astype(np.float32), acts)
        d = (A[-1] - e) * 2 / e.size; grads, _ = d126.backward(net, A, acts, d.astype(np.float32))
        opt.step(net, grads)
        if s % every == 0 and Xev is not None:
            log.append((s, eval_eps(eps_net_fn(net), *Xev)))
    return net, log
```

ACTUAL OUTPUT

```
network: [x_t (2 numbers), time embedding (32)] -> 256 -> 256 -> 2 (ReLU), Day 126's init_net,
    forward, backward and Adam;
loss: mean squared error between the true and the guessed eps; batches of 512 fresh ring points, t
    uniform on 1-1,000
evaluation: 20,000 fixed (x_0, t, eps) triples, t in 9 levels; the exact denoiser's error there is
    0.3290, the floor
    step    error  above the floor
   2,000   0.4102           0.0813
```

```
   4,000   0.3625           0.0335
   8,000   0.3645           0.0355
  12,000   0.3468           0.0178
  16,000   0.3585           0.0295
  20,000   0.3477           0.0187
  30,000   0.3419           0.0129
(196 s)
by t: error of the learned denoiser, and of the exact one (the floor), on the same triples
     t   learned     exact
     1    1.0044    0.9716
    10    0.5904    0.5645
    50    0.1029    0.0751
   100    0.1979    0.1761
   200    0.5094    0.5041
   300    0.5119    0.5104
   500    0.1440    0.1441
   700    0.0143    0.0138
  1000    0.0006    0.0001
Actual output from this lesson's run.
```

What it means

The floor is 0.3290, not 0: at middle noise levels x_t really does not say which way the noise pointed, so even the exact denoiser misses. The network fell from 0.4102 above it at step 2,000 to 0.3419 at step 30,000, 0.0129 above the floor. Progress was not smooth (0.3625 at 4,000, 0.3645 at 8,000), because each batch has fresh noise. By noise level, the largest gaps are at the smallest t: at t = 50, 0.1029 against 0.0751; there the noise is tiny, and guessing it means locating x_0 within the ring's 0.05 spread. At high t both errors approach zero, because x_t is almost all noise and the noise is almost x_t itself.

WHY CHOOSE THIS?

Noise prediction is a plain regression with a fixed target: no adversary to chase (compare Days 134–136).

WHY NOT THIS? (TRADEOFFS)

A small error per step is applied 1,000 times; section 5 shows what it does to the samples.

5. Sampling with the learned denoiser, and how much fresh noise to add

DEFINITION — start here, no prior knowledge assumed

Ho et al. (2020) used two settings for the fresh noise added at each step: $\sigma_t^2 = V_t$ (the posterior's own variance) or β_t (a little more). They note the first is optimal when the data is a single point and the second when the data is N(0, I). A third setting adds none. Each runs with both denoisers, three seeds, 2,000 samples.

REAL-WORLD SCENARIO

Shaking a sieve while sifting: shake too little and everything piles where it first landed; shake too much and fine grains bounce past where they belong.

IMPLEMENTATION

```
s2 = posterior(t)[2] if var == "posterior" else (b if var == "beta" else 0.0)
x = x + np.sqrt(s2) * rng.standard_normal(x.shape)
```

ACTUAL OUTPUT

```
2,000 samples each, seeds 0-2; fresh noise per step: beta-tilde_t (the posterior's own), beta_t (a little
    more), or none
 denoiser      noise  seed  modes  good share  spread   counts (the real ring's spread is 0.05)
  learned  posterior     0      8       0.976  0.0607   213 254 264 243 242 233 239 263
  learned  posterior     1      8       0.971  0.0592   224 248 263 235 234 237 214 287
  learned  posterior     2      8       0.976  0.0604   218 215 229 287 246 223 224 309
  learned       beta     0      8       0.963  0.0648   211 248 261 238 240 228 238 262
  learned       beta     1      8       0.964  0.0601   224 247 260 233 227 236 212 288
  learned       beta     2      8       0.966  0.0606   216 211 223 285 247 222 220 307
  learned       none     0      3       0.998  0.0228   1059 366 0 0 0 0 0 571
  learned       none     1      3       0.999  0.0202   1085 366 0 0 0 0 0 546
  learned       none     2      3       0.998  0.0231   1094 313 0 0 0 0 0 589
    exact  posterior     0      8       0.998  0.0455   232 239 266 244 250 261 263 240
    exact  posterior     1      8       0.995  0.0457   247 231 252 252 236 268 237 266
    exact  posterior     2      8       0.997  0.0460   241 219 226 285 240 253 256 273
    exact       beta     0      8       0.993  0.0496   232 239 267 240 247 259 264 238
    exact       beta     1      8       0.989  0.0497   246 229 250 251 236 265 236 264
    exact       beta     2      8       0.992  0.0501   239 212 230 283 238 252 254 276
    exact       none     0      8       1.000  0.0099   238 227 264 237 250 288 265 231
    exact       none     1      8       1.000  0.0099   265 234 262 245 231 241 247 275
    exact       none     2      8       1.000  0.0099   270 217 233 233 240 260 257 290
Actual output from this lesson's run.
```

What it means

With the exact denoiser, β_t gives the right spread (0.0496, 0.0497, 0.0501; real 0.05), V_t is too tight (0.0455), and no noise squeezes every mode to 0.0099 while still keeping all eight. Notice that good share rewards the tight samples (1.000 with no noise): a measure of 'on a mode' cannot see a lack of variety, which is why the spread is printed.

The learned denoiser is the other way round: its samples are too wide with either setting (0.0607, 0.0592, 0.0604 with V_t; up to 0.0648 with β_t), with a few stray points between modes (good share 0.976 at best against the exact 0.998). The network's remaining error adds width of its own. With no fresh noise it reached only three modes, the same three in every seed (seed 0's counts 1059 366 0 0 0 0 0 571), where the exact denoiser kept all eight. Without noise each sample follows the learned denoiser's average direction exactly; a likely reason, not tested here, is that small systematic errors in that direction then carry whole regions of starting points to the same few modes.

WHY CHOOSE THIS?

Fresh noise at every step lets the sampler recover from the denoiser's mistakes, and its size is a simple knob.

WHY NOT THIS? (TRADEOFFS)

The right size depends on the data and on how good the denoiser is: here β_t suited the exact denoiser, while the learned one was already too wide.

Interview preparation — technical questions

Q: Write the reverse step of a DDPM.

A: $x_{t-1} = (x_t - \beta_t/\sqrt{(1-A_t)}\ \varepsilon') / \sqrt{(1-\beta_t)} + \sigma_t\ z$, with no z on the last step.

Q: What does the network predict, and with what loss?

A: The noise ε inside x_t, by mean squared error, with t drawn at random each time.

Q: Why can't the error reach zero?

A: At middle noise levels x_t does not determine ε; here even the exact denoiser's error was 0.3290.

Q: β_t or V_t for the sampling noise?

A: Ho et al.: V_t is optimal for a single data point, β_t for N(0, I) data; on the ring, β_t gave the right spread with the exact denoiser (0.0497).

Q: How is the step number given to the network?

A: As sines and cosines of t at geometrically spaced frequencies, so near and far steps are both distinguishable.

Interview preparation — theoretical questions

Q: Why many small steps instead of one big one?

A: From high noise the best single guess is an average of all data: at t = 1,000 every exact guess was the ring's centre.

Q: Why compare a learned denoiser with an exact one?

A: It separates errors of the sampler from errors of the network; the exact one recovered eight modes, so the process is right.

Q: What does 'good share' miss?

A: Variety: samples with no fresh noise scored 1.000 while their spread was a fifth of the real one.

Q: What did removing the fresh noise do to the learned denoiser?

A: Its samples fell into 3 of the 8 modes, the same three in every seed.

Q: How is training a diffusion model different from training a GAN?

A: Training is one regression with a fixed target and no adversary, so it has no training-time collapse like Days 135–136's; sampling can still lose modes (three without fresh noise here), and it costs 1,000 network calls per sample.

Key takeaway and what's next

> **KEY TAKEAWAY**
> A diffusion model generates by repeating one small step: guess the noise in x_t, remove a little of it, add a little fresh noise back. With the exact denoiser that step recovered all eight modes of the ring, so the process works; a learned denoiser came close to the lowest possible error and recovered the modes too, a little too wide. The fresh noise is not decoration: without it, the learned denoiser's samples fell into three modes.

- Posterior formula against a million simulations: 0.03660 against 0.03634 (t = 50).
- One exact step from t = 1,000 gives the ring's centre for every point: a single jump cannot generate.
- 1,000 exact steps: 8 modes in every seed; spread 0.0455 to 0.0460 with V_t, 0.0496 to 0.0501 with β_t (real 0.0509).
- Learned denoiser: 0.3419 against the floor 0.3290 after 30,000 steps; samples 8 modes, spread about 0.06; with no fresh noise, 3 modes.

Suggested watch order, if starting from zero

- 1. Diffusion Models: DDPM | Generative AI Animated (Deepia) — an animated walk through DDPM, from the forward process to the simplified loss.
- 2. Diffusion Models | Paper Explanation | Math Explained (Outlier) — the paper's derivation of the reverse step and the noise-prediction loss.

Day 141 preview

- **Training a tiny diffusion model on images**: the same step, but with no exact denoiser to fall back on.